

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 14

TITLE:

Design and develop a Water Level Detection and Flood Alert System using Ultrasonic Sensor. The system measures water levels in rivers, tanks, or dams and provides early warning alerts during overflow conditions.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Water Level Detection and Flood Alert System is an IoT-based system designed to continuously monitor water levels in rivers, tanks, dams and other water-storage areas. The system uses an ultrasonic sensor to measure the distance between the sensor and the water surface.

A microcontroller such as an ESP8266/ESP32 processes the sensor readings and calculates the corresponding water level. Based on predefined threshold values, the system classifies the water condition as Normal, Warning or Critical.

When the water level reaches the critical or overflow limit, a red LED and buzzer are activated to provide an immediate warning. The water-level information can also be displayed on an OLED display and transmitted through Wi-Fi to an IoT/cloud platform for remote monitoring.

The proposed system provides a low-cost, real-time and automated flood warning solution that helps detect rising water levels early and supports flood prevention, disaster management and public safety.

Problem Statement:

Rising water levels in rivers, tanks, dams and reservoirs can lead to overflow and flooding, causing significant damage to human life, property, infrastructure and the environment. Conventional water-level monitoring methods may rely on manual inspection or periodic measurements, which can result in delayed detection of rapidly rising water levels.

The absence of a continuous and automated monitoring system makes it difficult to provide timely warnings before critical water levels are reached. Therefore, there is a need for a low-cost, real-time water-level detection system that can continuously monitor water levels, identify overflow conditions and provide immediate alerts to support early flood warning, disaster management and public safety.

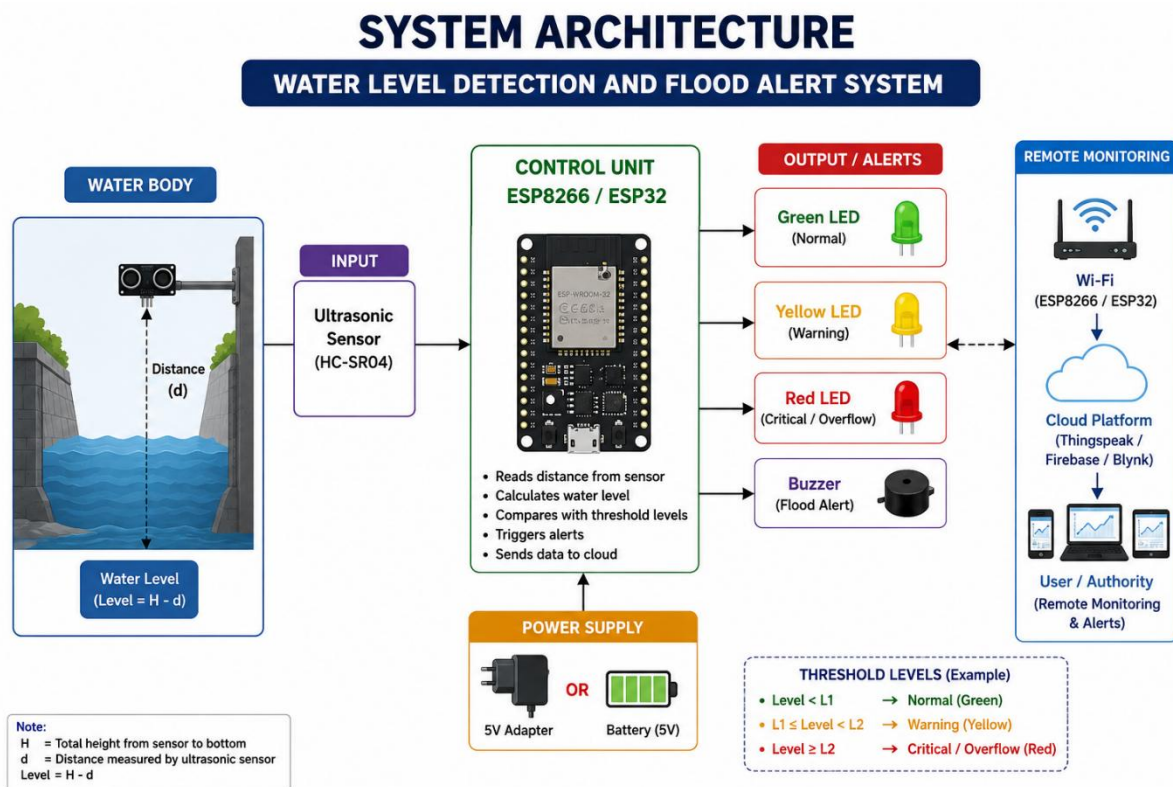
Proposed Solution:

- The proposed system uses an ultrasonic sensor and ESP8266/ESP32 microcontroller to continuously monitor water levels in rivers, tanks, dams and

reservoirs. The ultrasonic sensor is positioned above the water surface and measures the distance between the sensor and the water.

- The microcontroller processes the measured distance and calculates the corresponding water level. Based on predefined threshold values, the system classifies the condition as Normal, Warning or Critical.
- A green LED indicates a normal water level, while a yellow LED indicates a warning condition. When the water level reaches the critical or overflow limit, a red LED and buzzer are activated to provide an immediate flood alert.
- The measured water level can be displayed on an OLED display, while Wi-Fi connectivity can be used to transmit the data to an IoT/cloud platform for remote monitoring and data analysis.
- The proposed solution provides a low-cost, real-time and automated water-level monitoring system that helps detect rising water levels early and supports flood prevention, disaster management and public safety.

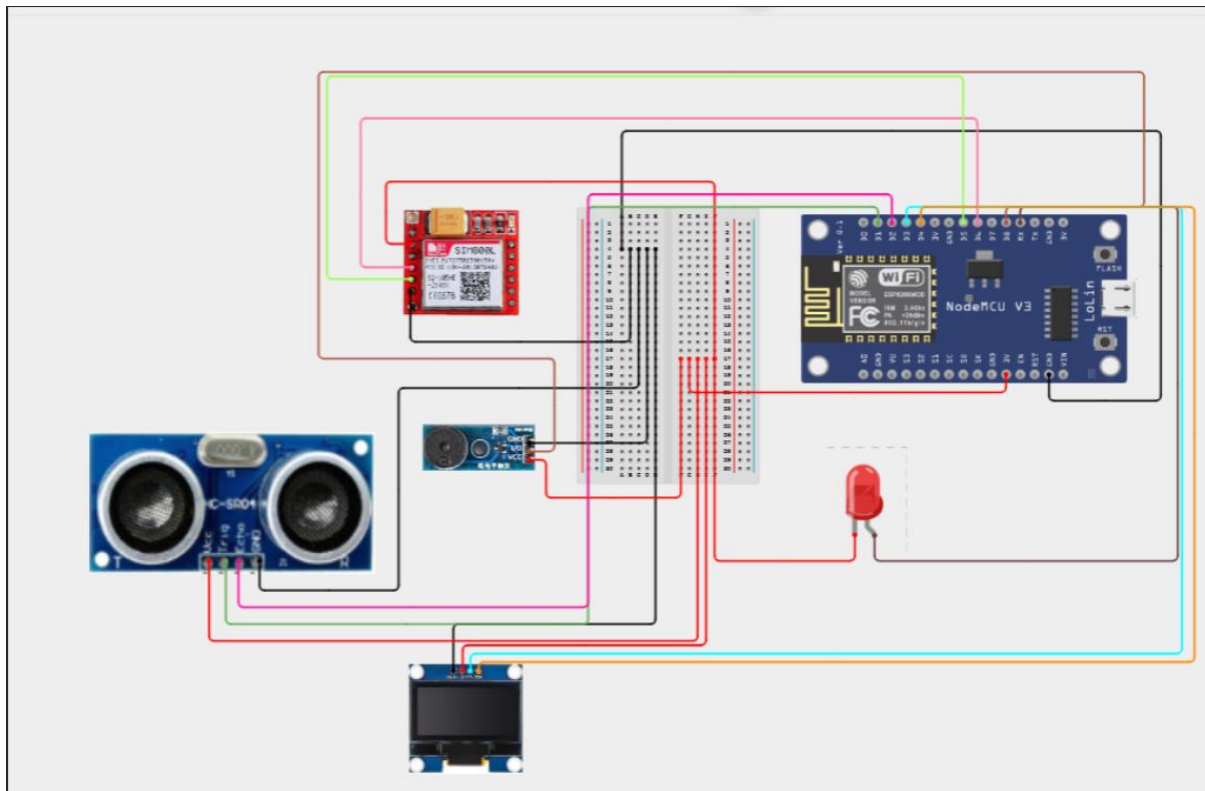
System Architecture:



Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
Ultrasonic Sensor (HC-SR04)	VCC	3.3V	Power supply
GSM Module	GND	GND	Ground
ESP8266	VIN/USB	5V USB	Hazardous indication
OLED Display	VCC	3.3V	Power supply
Green LED	Anode (+)	D5 (GPIO 14) through 220 Ω	Normal condition
Buzzer	Positive (+)	D4 (GPIO 2)	Flood alert

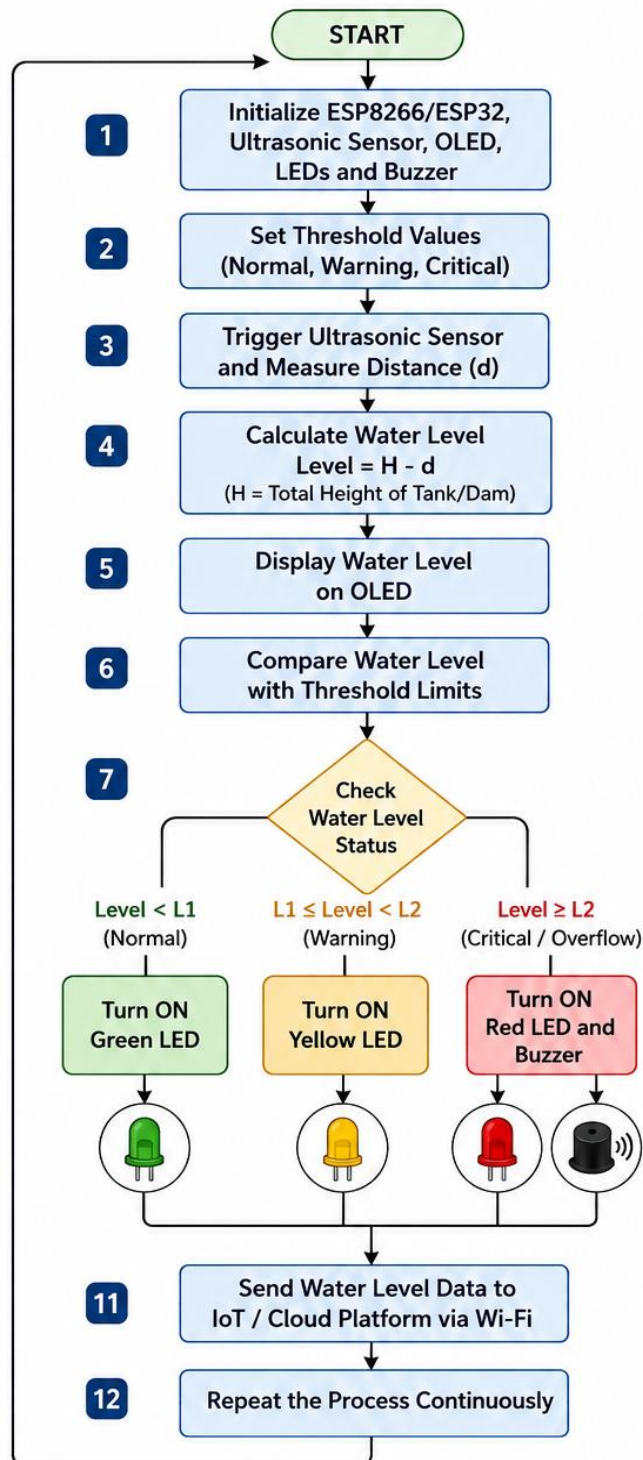
Circuit Design:



Algorithm:

ALGORITHM

WATER LEVEL DETECTION AND FLOOD ALERT SYSTEM



Where,

H = Total height from sensor to bottom
d = Distance measured by ultrasonic sensor
Level = H - d

THRESHOLD LEVELS (Example)

Level < L1 (Normal)	Safe Water Level Green LED ON
------------------------	----------------------------------

L1 ≤ Level < L2 (Warning)	Water Level Rising Yellow LED ON
------------------------------	-------------------------------------

Level ≥ L2 (Critical / Overflow)	Danger / Overflow Red LED + Buzzer ON
-------------------------------------	--

DESCRIPTION

- 1 System initializes all components.
- 2 Threshold values are set (L1 and L2).
- 3 Ultrasonic sensor measures distance (d) from sensor to water surface.
- 4 Water level is calculated using $\text{Level} = H - d$.
- 5 Water level is shown on OLED display.
- 6 System compares level with threshold limits.
- 7 Decision is made based on water level status.
- 8 If level is normal (Level < L1).
- 9 If level is in warning range (L1 ≤ Level < L2).
- 10 If level is critical or overflow (Level ≥ L2).
- 11 Data is sent to cloud for remote monitoring.
- 12 Process repeats forever for continuous monitoring.

Coding:

```
#include <ESP8266WiFi.h>

#include <SoftwareSerial.h>

#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====

// WIFI

// =====

#define WIFI_SSID    "Harish"
#define WIFI_PASSWORD "Harish@1234"

// =====

// ADAFRUIT IO

// =====

#define AIO_SERVER    "io.adafruit.com"
#define AIO_SERVERPORT 1883
#define AIO_USERNAME  "RISHI_44"
#define AIO_KEY       "aio_lmgi430fO7kh0Cq5iCaE2r1cGwPy"

// =====

// ULTRASONIC

// =====

#define TRIG_PIN 5
#define ECHO_PIN 4

// =====

// GSM

// =====

SoftwareSerial gsm(12, 14); // RX, TX

String phoneNumber = "+91XXXXXXXXXXXX";

// =====

// WATER LEVEL
```

```
// =====  
  
#define TANK_HEIGHT 100.0  
  
// Water level below 60 cm = DANGER  
  
#define DANGER_WATER_LEVEL 60.0  
  
// =====  
  
// MQTT  
  
// =====  
  
WiFiClient client;  
  
Adafruit_MQTT_Client mqtt(  
    &client,  
    AIO_SERVER,  
    AIO_SERVERPORT,  
    AIO_USERNAME,  
    AIO_KEY  
);  
  
// =====  
  
// FEEDS  
  
// =====  
  
Adafruit_MQTT_Publish waterFeed =  
    Adafruit_MQTT_Publish(  
        &mqtt,  
        AIO_USERNAME "/feeds/water-level"  
    );  
  
Adafruit_MQTT_Publish rainfallFeed =  
    Adafruit_MQTT_Publish(  
        &mqtt,  
        AIO_USERNAME "/feeds/rainfall"  
    );  
  
Adafruit_MQTT_Publish alertFeed =  
    Adafruit_MQTT_Publish(  

```

```

    &mqtt,
    AIO_USERNAME "/feeds/alert"
);
Adafruit_MQTT_Publish smsFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/sms-status"
);
// =====
// STATE
// =====
bool previousDanger = false;
// =====
// WIFI CONNECTION
// =====
void connectWiFi() {
    if (WiFi.status() == WL_CONNECTED) {
        return;
    }
    Serial.println();
    Serial.print("Connecting WiFi");
    WiFi.begin(
        WIFI_SSID,
        WIFI_PASSWORD
    );
    int attempts = 0;
    while (
        WiFi.status() != WL_CONNECTED &&
        attempts < 30
    ) {

```



```
    delay(500);

    Serial.print(".");

    attempts++;

}

Serial.println();

if (WiFi.status() == WL_CONNECTED) {

    Serial.println("WiFi connected");

} else {

    Serial.println("WiFi connection failed");

}

}

// =====

// ADAFRUIT CONNECTION

// =====

void MQTT_connect() {

    if (mqtt.connected()) {

        return;

    }

    Serial.print("Connecting Adafruit IO");

    int8_t ret;

    while ((ret = mqtt.connect()) != 0) {

        Serial.print(".");

        delay(2000);

    }

    Serial.println();

    Serial.println("Adafruit IO connected");

}

// =====

// ULTRASONIC

// =====
```

```
float getDistance() {  
    digitalWrite(  
        TRIG_PIN,  
        LOW  
    );  
    delayMicroseconds(3);  
    digitalWrite(  
        TRIG_PIN,  
        HIGH  
    );  
    delayMicroseconds(10);  
    digitalWrite(  
        TRIG_PIN,  
        LOW  
    );  
    unsigned long duration =  
        pulseIn(  
            ECHO_PIN,  
            HIGH,  
            30000  
        );  
    if (duration == 0) {  
        return -1;  
    }  
    float distance =  
        duration * 0.0343 / 2.0;  
    return distance;  
}  
  
// =====  
  
// WATER LEVEL
```

```
// =====  
  
float getWaterLevel(  
    float distance  
) {  
    float waterLevel =  
        TANK_HEIGHT - distance;  
    if (waterLevel < 0) {  
        waterLevel = 0;  
    }  
    if (waterLevel > TANK_HEIGHT) {  
        waterLevel = TANK_HEIGHT;  
    }  
    return waterLevel;  
}  
  
// =====  
  
// RAINFALL FROM WATER LEVEL  
  
// =====  
  
float calculateRainfall(  
    float waterLevel  
) {  
    if (waterLevel < 10) {  
        return 110;  
    }  
    else if (waterLevel < 20) {  
        return 100;  
    }  
    else if (waterLevel < 30) {  
        return 80;  
    }  
    else if (waterLevel < 40) {
```

```
    return 70;
}
else if (waterLevel < 50) {
    return 60;
}
else if (waterLevel < 60) {
    return 50;
}
else if (waterLevel < 70) {
    return 40;
}
else if (waterLevel < 80) {
    return 30;
}
else if (waterLevel < 90) {
    return 20;
}
else {
    return 10;
}
}

// =====
// SEND SMS
// =====
//
// We DO NOT check GSM response.
// When danger occurs, we send the SMS command and
// consider it "SMS SENT" for the dashboard.
//
// =====
```

```

void sendSMS(
    String message
) {
    Serial.println();
    Serial.println(
        "=====
    );
    Serial.println(
        "    FLOOD SMS ALERT"
    );
    Serial.println(
        "=====
    );
    // SMS text mode
    gsm.println(
        "AT+CMGF=1"
    );
    delay(500);
    // Mobile number
    gsm.print(
        "AT+CMGS=\""
    );
    gsm.print(
        phoneNumber
    );
    gsm.println(
        "\""
    );
    delay(1000);
    // Message

```

```
gsm.print(
  message
);
delay(500);
// CTRL + Z
gsm.write(26);
Serial.println(
  "SMS SENT"
);
}
// =====
// UPLOAD DATA
// =====

void uploadData(
  float waterLevel,
  float rainfall,
  String alertStatus
) {
  connectWiFi();
  if (
    WiFi.status() != WL_CONNECTED
  ) {
    return;
  }
  MQTT_connect();
  // Water level
  waterFeed.publish(
    waterLevel
  );
  // Rainfall
```

```
rainfallFeed.publish(  
    rainfall  
);  
// Alert  
alertFeed.publish(  
    alertStatus.c_str()  
);  
Serial.println();  
Serial.println(  
    "Data uploaded to Adafruit IO"  
);  
Serial.print(  
    "Water Level : "  
);  
Serial.print(  
    waterLevel  
);  
Serial.println(  
    " cm"  
);  
Serial.print(  
    "Rainfall  : "  
);  
Serial.print(  
    rainfall  
);  
Serial.println(  
    " mm"  
);  
Serial.print(  

```

```
    "Alert    : "  
  );  
  Serial.println(  
    alertStatus  
  );  
}  
// =====  
// SETUP  
// =====  
void setup() {  
  Serial.begin(  
    115200  
  );  
  gsm.begin(  
    9600  
  );  
  pinMode(  
    TRIG_PIN,  
    OUTPUT  
  );  
  pinMode(  
    ECHO_PIN,  
    INPUT  
  );  
  digitalWrite(  
    TRIG_PIN,  
    LOW  
  );  
  Serial.println();  
  Serial.println(  

```



```

    "=====
);
Serial.println(
    "    SMART FLOOD ALERT SYSTEM"
);
Serial.println(
    "=====
);
connectWiFi();
if (
    WiFi.status() == WL_CONNECTED
) {
    MQTT_connect();
}
Serial.println();
Serial.println(
    "SYSTEM READY"
);
}
// =====
// LOOP
// =====

void loop() {
    // -----
    // WIFI
    // -----
    connectWiFi();
    // -----
    // ADAFRUIT
    // -----

```

```
if (
    WiFi.status() == WL_CONNECTED
) {
    MQTT_connect();
}
// -----
// ULTRASONIC
// -----
float distance =
    getDistance();
if (distance < 0) {
    Serial.println(
        "Ultrasonic sensor error"
    );
    delay(2000);
    return;
}
// -----
// WATER LEVEL
// -----
float waterLevel =
    getWaterLevel(
        distance
    );
// -----
// RAINFALL
// -----
float rainfall =
    calculateRainfall(
        waterLevel
```

```
);  
// -----  
// DANGER LOGIC  
// -----  
bool currentDanger = false;  
if (  
    waterLevel < DANGER_WATER_LEVEL  
) {  
    currentDanger = true;  
}  
// -----  
// SERIAL  
// -----  
Serial.println();  
Serial.println(  
    "-----"  
);  
Serial.print(  
    "Distance : "  
);  
Serial.print(  
    distance,  
    2  
);  
Serial.println(  
    " cm"  
);  
Serial.print(  
    "Water Level : "  
);
```

```
Serial.print(  
    waterLevel,  
    2  
);  
Serial.println(  
    " cm"  
);  
Serial.print(  
    "Rainfall : "  
);  
Serial.print(  
    rainfall  
);  
Serial.println(  
    " mm"  
);  
// =====  
// DANGER  
// =====  
if (currentDanger) {  
    Serial.println();  
    Serial.println(  
        "***** DANGER *****"  
    );  
    // Send DANGER to Adafruit  
    uploadData(  
        waterLevel,  
        rainfall,  
        "DANGER"  
    );  
}
```

```

// -----
// SEND SMS ONLY ONCE
// -----
if (!previousDanger) {
    String message =
        "FLOOD ALERT! "
        "Water level = "
        + String(
            waterLevel,
            1
        )
        + " cm. "
        "Immediate action required.";
    // Send SMS command
    sendSMS(
        message
    );
    // IMPORTANT:
    // Do not check GSM.
    // Directly show SMS SENT.
    smsFeed.publish(
        "SMS SENT"
    );
    Serial.println(
        "Adafruit SMS STATUS = SMS SENT"
    );
}
}
// =====
// SAFE

```

```

// =====
else {
  Serial.println();
  Serial.println(
    "***** SAFE *****"
  );
  // Send SAFE to Adafruit
  uploadData(
    waterLevel,
    rainfall,
    "SAFE"
  );
  // No SMS
  smsFeed.publish(
    "SMS NOT SENT"
  );
}
// -----
// SAVE STATE
// -----
previousDanger =
  currentDanger;
// -----
// WAIT
// -----
Serial.println();
Serial.println(
  "Next update in 30 seconds..."
);
delay(30000);}

```

System Working:

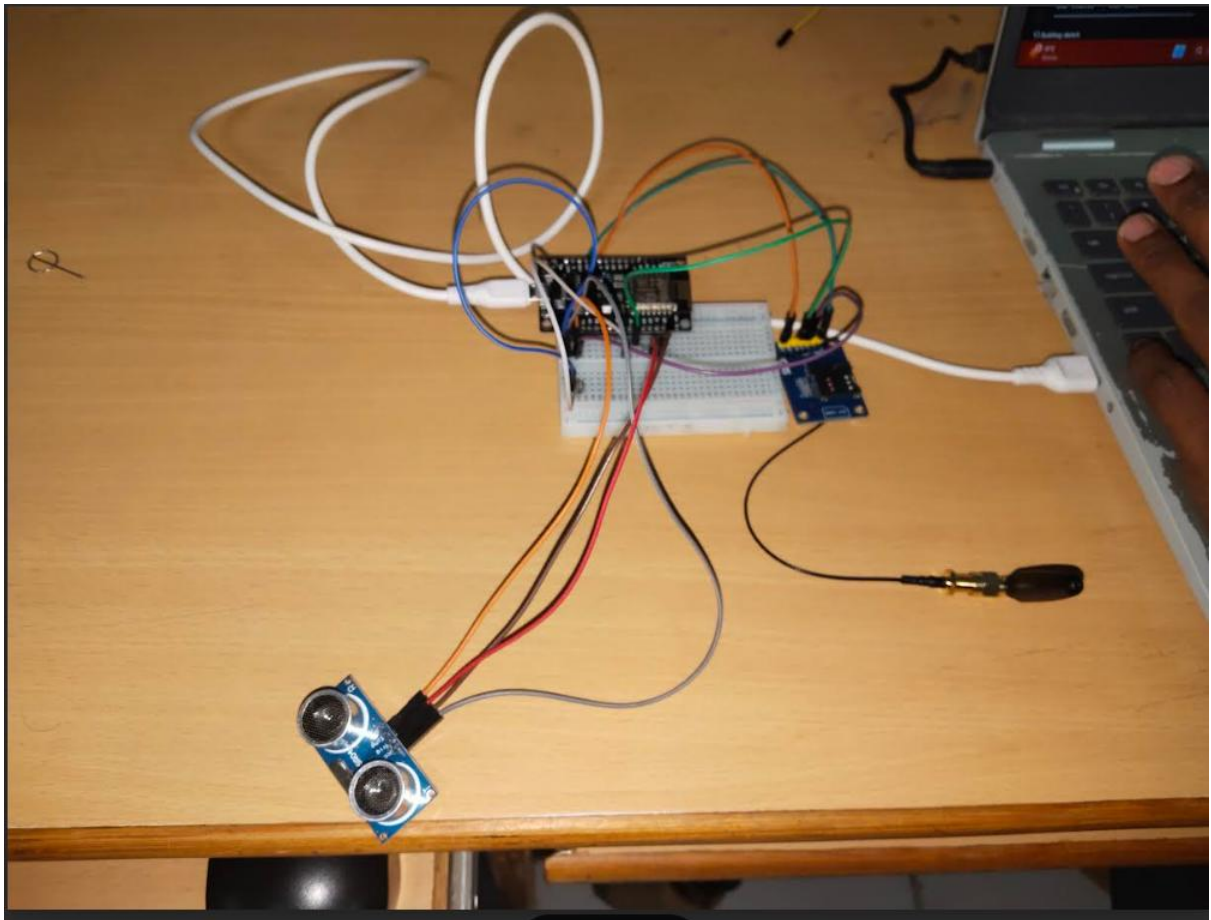
- The Water Level Detection and Flood Alert System uses an ultrasonic sensor mounted above the water surface to continuously measure the distance between the sensor and the water. The ESP8266/ESP32 processes the measured distance and calculates the corresponding water level.
- The water level is displayed on an OLED display and compared with predefined threshold values. Under normal conditions, the green LED is activated, while a rising water level activates the yellow LED.
- When the water level reaches the critical or overflow limit, the red LED and buzzer are activated to provide an immediate flood warning. The ESP8266/ESP32 can also transmit the water-level data through Wi-Fi to an IoT/cloud platform for remote monitoring.
- The system continuously repeats this process, providing a real-time and automated early-warning solution for flood and overflow conditions.

Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	Ultrasonic Sensor	HC-SR04	1	Rs. 80
3	Buzzer	Active Buzzer	1	Rs. 20
4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	Rs. 50
6	OLED Display	0.96" I ² C, 128×64	1	Rs. 150
		Total		Rs. 1250

Prototype Implementation:

- The prototype was implemented using an ESP8266/ESP32 microcontroller, ultrasonic sensor, OLED display, green, yellow and red LEDs, and a buzzer. The ultrasonic sensor was positioned above the water surface to measure the distance between the sensor and the water. The microcontroller processes the sensor data and calculates the corresponding water level.
- The measured water level is displayed on the OLED display and compared with predefined threshold values. The green LED indicates a normal water level, while the yellow LED indicates a warning condition. When the water level reaches the critical limit, the red LED and buzzer are activated to provide an immediate flood alert.
- The ESP8266/ESP32 can also transmit the water-level data through Wi-Fi to an IoT/cloud platform for remote monitoring. The prototype continuously updates the water-level readings and provides alerts whenever a critical condition is detected.



Results and Performance:

- The developed Water Level Detection and Flood Alert System successfully monitored the water level using the ultrasonic sensor and ESP8266/ESP32. The sensor continuously measured the distance to the water surface, and the microcontroller calculated and displayed the water level on the OLED display. The system successfully classified the water condition into Normal, Warning and Critical levels based on predefined thresholds.
- The green, yellow and red LEDs provided clear visual indications, while the buzzer was activated during critical water levels to provide an immediate flood alert. The system demonstrated real-time and continuous monitoring and can be connected to an IoT/cloud platform for remote monitoring. Overall, the prototype provides a low-cost, reliable and automated solution for early flood detection and water-level monitoring.

